

Estructuras de Datos (GPD4402) - Tarea 6: Grafos, Ordenamiento por Division y Separacion de Intereses

Documento de reflexion

Igal Shturman Poplawsky - Matricula 18139

1. Sobre la propiedad de la memoria

En mi Graph el unico dueño es el grafo. Mi destructor es así: primero recorro `_edges` y hago delete de cada arista, y después recorro `_nodes` y hago delete de cada nodo. Ni Node ni Edge liberan nada, solo apuntan. Esto es a propósito porque la misma arista está guardada en la lista de vecinos de sus dos nodos (cuando hago `AddEdge` la meto en `_edges` y después la agrego con `AddNeighbor` a los dos nodos). Si cada nodo liberara sus aristas pasaría esto: muere el nodo A y su destructor borra la arista A-B, después muere el nodo B y su destructor intenta borrar la misma arista otra vez. Eso es un double free: la segunda liberación es sobre memoria que ya se regresó al sistema, y el programa se cae o se corrompe en silencio. Por eso centralice todo en el grafo, cada cosa se libera una sola vez.

2. Sobre la separacion de intereses

Mi Graph.cs no tiene using UnityEngine porque es lógica pura. Si lo dejara tendría dos problemas concretos: primero, ese archivo ya no compilaría fuera de Unity (en un proyecto de consola de C#), o sea que no podría reutilizarlo en un servidor o en otro juego; segundo, no podría probar el recorrido de forma automática, porque una prueba solo puede revisar valores que le regresan, no lo que salió con `Debug.Log` en la consola. Además, si quisiera pintar el recorrido y guardarlo en un archivo tendría que recorrer dos veces o modificar la clase. Mi corrutina pudo animar sin que el grafo supiera de colores ni de tiempo porque mi DFS y mi BFS ya no imprimen: regresan una `List<Node>` con el orden. La corrutina solo reproduce esa lista poco a poco con `WaitForSeconds` y pinta el `GameObject` de cada nodo usando un diccionario `Node-GameObject`. El grafo nunca se entera de que existe la pantalla.

3. Sobre Quick Sort

En mi código el pivote es el último elemento (`arreglo[fin]`) con partición de Lomuto. Si le paso un arreglo que ya está ordenado le pasa lo mismo que si eligiera siempre el primero: el pivote siempre es el más grande, así que la partición queda de 0 elementos de un lado y $n-1$ del otro. Eso se repite en cada nivel, la recursión baja de uno en uno y se vuelve $O(n^2)$, además de que hace muchas llamadas recursivas. Mi MergeSort con el mismo arreglo no sufre nada porque siempre parte a la mitad sin fijarse en los valores: hace el

mismo trabajo ordenado o desordenado, $O(n \log n)$, solo que necesita un arreglo temporal para mezclar.

Referencias

- Unity Technologies. (s.f.). Coroutines. <https://docs.unity3d.com/Manual/Coroutines.html>
- VisuAlgo. (s.f.). Graph Traversal. <https://visualgo.net/es/dfsbfbs>
- VisuAlgo. (s.f.). Sorting. <https://visualgo.net/es/sorting>